ANN benchmark report: HNSW, IVF-Flat and IVF-PQ on SIFT-1M and GloVe-100

Recall means recall@10 against exact search. Speed is queries per second, one query at a time on one thread. Index builds used 20 threads. Every number comes from my result files (*Report_analysis.ipynb* regenerates all tables and figures).

1. At recall 0.95, which index is fastest? Does it change between datasets?

HNSW is fastest on both datasets, IVF-Flat is second, and IVF-PQ never gets to 0.95.

Dataset	Index (best setting)	QPS at 0.90	QPS at 0.95	Vectors compared at 0.95	Speed-up vs exact
SIFT	HNSW, M=16, efC=200	12,259	7,837	1,092 (0.11%)	238x
SIFT	IVF-Flat, nlist=4096	3,425	2,084	15,140 (1.51%)	63x
SIFT	IVF-PQ (best code size)	not reached	not reached (max 0.73)	-	-
SIFT	Exact scan (1 thread)	33	33	1,000,000	1x
GLOVE	HNSW, M=32, efC=100	1,278	466	13,594 (1.15%)	6x
GLOVE	IVF-Flat, nlist=1024, spherical	750	317	108,302 (9.15%)	4x
GLOVE	IVF-PQ (best code size)	not reached	not reached (max 0.85)	-	-
GLOVE	Exact scan (20 threads*)	72	72	1,183,514	1x

*I did not have a single-thread exact timing for GloVe, so I used the 20-thread one. That flatters exact search, so the GloVe speed-ups are lower bounds and can't be compared with SIFT's. Table values are interpolated at exactly the recall target.

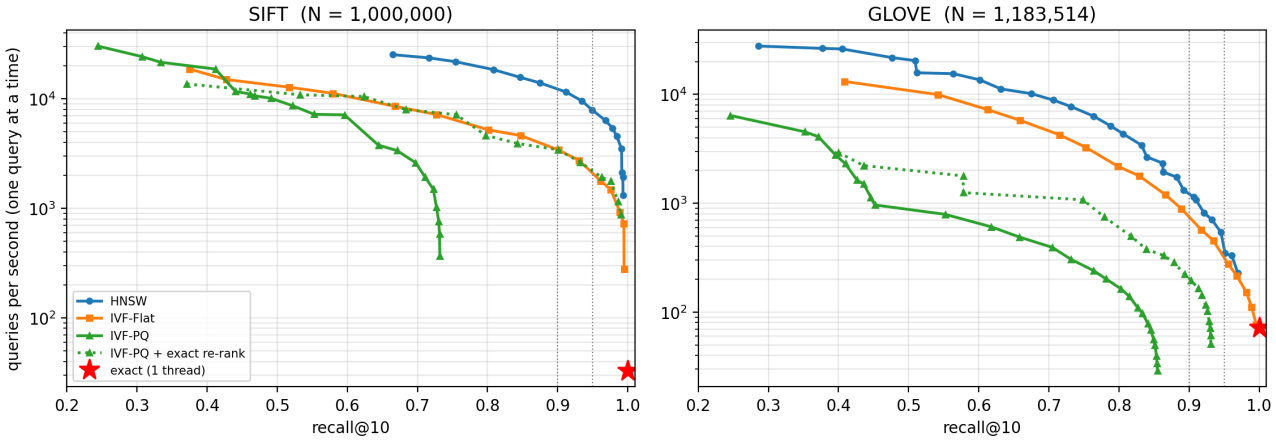


Figure 1. Best speed-vs-recall curve of each index, and exact search (star). The dotted green curve adds exact re-ranking, which stores the full vectors again.

- **The ranking holds, the gap doesn't.** On SIFT, HNSW does 7,837 QPS against 2,084 for IVF-Flat (3.8x). On GloVe it is 466 against 317 (1.5x).
- **Tuning can flip the answer on GloVe.** The default HNSW (M=16) stalls at recall 0.937 and never reaches 0.95, while the default IVF-Flat does (9.4 ms). HNSW only wins after I raise M to 32.
- **Why HNSW is faster:** it compares far fewer vectors per query (1,092 against 15,140 on SIFT). It follows links that lead towards the query. IVF-Flat has to scan every vector in each cell it probes, and a neighbour just across a cell edge is only found by probing that cell too.
- **Why the latency gap is smaller than that** (3.8x against 14x in vectors): HNSW jumps around memory (about 117 ns per distance) while IVF-Flat reads its lists in order (about 32 ns).
- **Why GloVe is harder:** its neighbours are barely closer than other points, so cells separate them badly (IVF-Flat compares 9% of the data, SIFT 1.5%) and HNSW needs more links (M=16 stalls at 0.94, M=8 at 0.84).
- **IVF-Flat's default k-means is also bad on GloVe:** the lists are very uneven (imbalance 15.7, ideal 1.0, SIFT 1.1). Spherical k-means gets it to 1.18 and makes search 3x faster. I think the mean of a loose cluster is shorter than the mean of a tight one, which biases the inner-product assignment, but I didn't test that.

2. Build time and memory. What does PQ save and what does it give up?

Index	SIFT build (s)	SIFT bytes/vector	SIFT max recall	GloVe build (s)	GloVe bytes/vector	GloVe max recall
HNSW, M=16, efC=100	32.5	656	0.99	39.6	544	0.94
HNSW, M=32	38.3	784	0.99	53.5	672	0.97
HNSW, M=16, efC=200	55.8	656	0.99	71.7	544	0.95
IVF-Flat, nlist=1024	14.6	521	0.99	13.3	408	0.99
IVF-Flat, nlist=4096	193.0	522	0.99	138.2	409	0.99
IVF-PQ, small code (M=8 | 10)	28.9	16.7	0.39	22.5	18.4	0.24
IVF-PQ, base code (M=16 | 20)	34.0	24.7	0.57	30.9	28.4	0.47
IVF-PQ, large code (M=32 | 50)	47.3	40.7	0.73	55.3	58.4	0.85
IVF-PQ, base code + exact re-rank	34.2	537	0.99	31.1	428	0.93
Exact scan	0	512	1.00	0	400	1.00

Table 2. Build time, index size per vector (whole index file / number of vectors) and the highest recall each setting reached (sweeps stop at 0.99). The full table is in the notebook.

- **HNSW:** every vector is inserted by searching the graph built so far, so build time grows with efConstruction (13 s to 56 s for 40 to 200 on SIFT). Memory is the raw vector plus links: 656 bytes on SIFT, of which 144 are graph.
- **IVF-Flat:** no memory saving (raw vector plus an 8-byte id). The build is k-means, which grows with the number of cells: 15 s at 1024 cells, 193 s at 4096. The fastest IVF-Flat setting therefore costs 3.5x the build of the fastest HNSW.
- **What PQ saves:** each vector becomes M one-byte codes: 17 to 41 bytes on SIFT (13x to 31x smaller) and 18 to 58 on GloVe. Ten million SIFT vectors would fit in about 0.25 GB.
- **What it gives up, recall:** distances to codes are approximate, so some true neighbours rank too low however many cells you probe. The ceiling is 0.39 / 0.57 / 0.73 on SIFT and 0.24 / 0.47 / 0.85 on GloVe (small / base / large code). More bytes help, and so does encoding residuals (+0.02 SIFT, +0.05 GloVe).
- **It isn't faster either.** On the same cells, at equal recall, PQ is 1.2x to 2.4x slower than IVF-Flat on SIFT and 0.9x to 1.8x on GloVe. My guess is the look-up tables get rebuilt for every probed cell, but I didn't test it.
- **Re-ranking** with the exact vectors fixes recall (SIFT reaches 0.99) but brings memory back to 537 bytes per vector, more than IVF-Flat. PQ only saves memory if I accept the ceiling or keep the full vectors somewhere else, such as on disk.

3. What do Parts 3 and 4 show, and when could the headline numbers mislead?

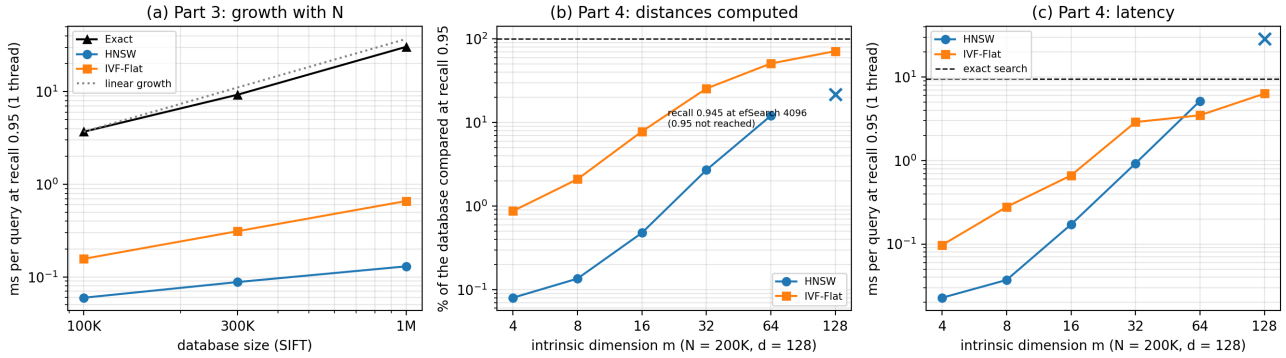


Figure 2. (a) Part 3: cost at recall 0.95 on SIFT at 100K, 300K, 1M vectors. (b, c) Part 4: only the intrinsic dimension m of generated data changes (N = 200K, d = 128). The cross is HNSW at m = 128, which ended at recall 0.945 and never reached 0.95.

Part 3, scaling (SIFT, 100K to 1M):

- Exact search got 8.2x slower, IVF-Flat 4.2x, HNSW 2.2x.
- Why: exact search touches every vector. IVF-Flat touches a share that shrinks as cells get smaller (4.2% of cells probed down to 2.0%). HNSW's walk gets longer only slowly.
- So the speed-up over exact search grows with N: HNSW 62x to 234x, IVF-Flat 24x to 46x. HNSW's build time grows faster than the data (20x for 10x vectors).
- My four predictions held, but I wrote the hypothesis after a first exploratory run, so it wasn't a blind test.

Part 4, geometry (intrinsic dimension 4 to 128, N fixed):

- The contrast between the 10th neighbour and an average point falls from 9.7 to 1.26.
- IVF-Flat's cost rises from 0.9% to 72% of the data, because the true neighbours spread over many cells and the nearest centroids stop identifying them.
- HNSW rises from 0.08% to 12% at m = 64. At m = 128 it only reaches recall 0.945, after 22% of the data and 29 ms, **three times slower than exact search** (9.5 ms). Its walk has too little gradient to follow.
- Re-making m = 16 with another seed changed the cost by only 0.3% (HNSW) and 1.9% (IVF-Flat), so the effect of m is real.
- 6 of 8 predictions held. The two misses were my errors: IVF-Flat at m = 128 is still 1.5x faster than exact search, and my check counted "not reached" as infinitely expensive although HNSW still needs fewer distances than IVF-Flat at equal recall (22% against 81%).

So headline numbers can mislead when:

- **the size is different:** HNSW is 234x faster than exact search at 1M but 62x at 100K, and I never measured above 1M.
- **the data is different:** SIFT is easy. The same N, dimension and memory cost IVF-Flat 0.9% or 72% of the data depending only on structure, and HNSW can lose to exact search.
- **vectors compared are quoted instead of time:** at m = 64 HNSW compares 4.2x fewer vectors than IVF-Flat but is 1.5x slower (214 ns per distance against 34).
- **only one recall level is shown:** ceilings (PQ, HNSW on GloVe) stay hidden.
- **defaults are compared:** the GloVe ranking flips with tuning (question 1).

4. 150,000 or 10 million vectors, 100 ms per query, 16 GB RAM: would I use ANN?

N	Option	Latency (ms)	Memory (GB)	Build (s)	Fits 100 ms and 16 GB?
150K	Exact scan, 1 thread	5.18	0.08	0	yes
150K	HNSW M=16	0.07	0.10	3	yes
10M	Exact scan, 1 thread	304	5.12	0	no: too slow
10M	Exact scan, 20 threads, one query	234	5.12	0	no: too slow
10M	IVF-Flat, nlist ~ sqrt(N)	2.77	5.21	112	yes
10M	HNSW M=16, efC=100	0.29	6.56	624	yes
10M	HNSW M=32 (GloVe-like data)	4.67	6.72	1088	yes
10M	IVF-PQ M=16	-	0.25	-	no: recall stops at 0.57

Table 3. Built from my measured exact timings on SIFT (3.7, 9.3, 30.4 ms at 100K, 300K, 1M; one thread) and the index results. 150K is interpolated. The 10M rows are estimates, 10x beyond what I measured (power-law fits on three points; the GloVe-like row assumes SIFT's growth). Memory ignores operating-system overhead.

- **150K: no ANN.** Exact search takes about 5.2 ms, 19x under budget, with 77 MB, recall exactly 1.0 and nothing to build or tune. ANN would be faster, but nothing here needs it.
- **10M: yes.** Exact search would take about 304 ms (30 ns per vector, linear in N), three times over budget. Twenty threads speed up a single query only 1.3x, and batching needs thousands of queries at once, so neither meets a per-query budget.
- **RAM isn't the problem:** the raw vectors are 5.1 GB. So I wouldn't use PQ, and pure PQ can't reach 0.95 anyway.
- **My choice: IVF-Flat** with about 3,200 cells (sqrt of N), nprobe tuned on held-out queries until recall hits 0.95 (I expect about 30), and spherical k-means if the data is angular. Estimate: 2.8 ms, 5.2 GB, about 2 minutes to build. The budget is loose, so I'd pick the one with no recall ceiling and the cheapest build. Even Part 4's worst case would take about 218 ms, faster than exact search.
- **I'd switch to HNSW** (M=16, efConstruction 100 to 200, efSearch 60 to 100) only for sub-millisecond latency or a high query rate: about 0.3 ms, 6.6 GB, 10 minutes to build. For GloVe-like data it needs M=32 and efSearch in the hundreds (about 5 ms, 6.7 GB).
- **Caveat:** the 10M latencies are extrapolated, so I'd check them on a 1 to 2 million sample of the real data first. If the data is as unstructured as Part 4's m = 128 case, neither index reaches 95% recall within 100 ms on one thread.

5. Two ways my benchmark could mislead, and whether I tested them

- **Timing noise.** Latency is wall-clock time on a shared 20-thread machine. Repeats of the same timing differ by a median of 6 to 18%. Rebuilding the same HNSW gave identical distance counts but latencies 16% and 18% apart, so that's timing noise. Exact search at 1M took 44.7 ms in one run and 30.4 ms in another. Differences under about 20% mean nothing: "efConstruction=200 is fastest" is 0.13 against 0.23 ms but only 6% fewer distances.
- *Tested partly:* the rebuild check, more repeats (3 to 5), interpolating at exactly the target recall, and counting distances as a hardware-independent cross-check. *Not tested:* other machines or a busy server, and ANN search in batches or with several threads.
- **Choosing settings on the queries I report on.** The best settings were picked from many on the same 10,000 test queries that give the reported recall, and the queries come from the same distribution as the database. That makes the results a bit optimistic.
- *Not tested:* I had no held-out query set, so I can't say how big the effect is. I expect it to be small with 10,000 queries, but that is only an expectation. The database side of the question was tested in Part 4, which showed that data structure matters far more than tuning.